

1. Histograms, Boxplots, Outliers

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)

df.hist(figsize=(10,8), bins=30)
plt.show()

df.plot(kind='box', subplots=True, layout=(3,3), figsize=(10,8))
plt.show()

print("Outliers:")
for c in df.columns:
    Q1 = df[c].quantile(0.25)
    Q3 = df[c].quantile(0.75)
    IQR = Q3 - Q1
    out = df[(df[c] < Q1-1.5*IQR) | (df[c] > Q3+1.5*IQR)]
    print(c,":",len(out))
```

2. Correlation + HeatMap + PairPlot

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()

sns.pairplot(df.sample(300))
plt.show()
```

3. PCA (Iris 4D → 2D)

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

iris = load_iris()
X,y = iris.data, iris.target

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

```
print(pca.explained_variance_ratio_)
```

4. Find-S Algorithm

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("training_data.csv")
print(df)
```

```
X = np.array(df.iloc[:, :-1])
y = np.array(df.iloc[:, -1])
```

```
h = X[y=='Yes'][0]
```

```
for x in X[y=='Yes']:
    for i in range(len(h)):
        if h[i] != x[i]:
            h[i] = '?'
```

```
print("Hypothesis:", h)
```

5. KNN (Manual)

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
```

```
np.random.seed(1)
x = np.random.rand(100)
```

```
tr_x, ts_x = x[:50], x[50:]
tr_y = ["Class1" if i <= 0.5 else "Class2" for i in tr_x]
```

```
for k in [1, 2, 3, 4, 5, 20, 30]:
    preds = []
    for p in ts_x:
        d = abs(tr_x - p)
        idx = d.argsort()[:k]
        lab = [tr_y[i] for i in idx]
        preds.append(Counter(lab).most_common(1)[0][0])
    print(k, preds.count("Class1"), preds.count("Class2"))
```

```
plt.scatter(tr_x, [0]*50, c=['b' if l=='Class1' else 'r' for l in tr_y])
```

```
plt.scatter(ts_x,[1]*50,c=['b' if p=='Class1' else 'r' for p in preds])
plt.show()
```

6. Locally Weighted Regression

```
import numpy as np
import matplotlib.pyplot as plt

X = np.linspace(0,10,50)
y = np.sin(X)

X_mat = np.c_[np.ones(50),X]

def lwr(x,X,y,t):
    w = np.exp(-((X-x)**2).sum(axis=1)/(2*t*t))
    W = np.diag(w)
    theta = np.linalg.inv(X.T.dot(W).dot(X)).dot(X.T).dot(W).dot(y)
    return x.dot(theta)

x_test = np.linspace(0,10,100)
pred = [lwr([1,i],X_mat,y,0.5) for i in x_test]

plt.scatter(X,y)
plt.plot(x_test,pred)
plt.show()
```

7. Linear + Polynomial Regression

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)

X = df[['AveRooms']]
y = data.target

model = LinearRegression().fit(X,y)

plt.scatter(X,y)
plt.plot(X,model.predict(X))
plt.show()

url="https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
cols=["mpg","c","disp","hp","wt","acc","yr","orig"]
df2 = pd.read_csv(url,sep='\s+',names=cols).dropna()
```

```
X2 = df2[['disp']]
y2 = df2['mpg']

X_poly = PolynomialFeatures(2).fit_transform(X2)
model2 = LinearRegression().fit(X_poly,y2)

plt.scatter(X2,y2)
plt.scatter(X2,model2.predict(X_poly))
plt.show()
```

8. Decision Tree

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier, plot_tree

data = load_breast_cancer()
X,y = data.data, data.target

clf = DecisionTreeClassifier().fit(X,y)

print(data.target_names[clf.predict(X[:1])[0]])

plot_tree(clf)
plt.show()
```

9. Naive Bayes (Olivetti)

```
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

data = fetch_olivetti_faces()
X,y = data.data, data.target

X_tr,X_ts,y_tr,y_ts = train_test_split(X,y,test_size=0.2)

model = GaussianNB().fit(X_tr,y_tr)

print("Accuracy:",model.score(X_ts,y_ts)*100)

for i in range(6):
    plt.subplot(2,3,i+1)
    plt.imshow(X_ts[i].reshape(64,64),cmap='gray')
    plt.title(str(y_ts[i])+" "+str(model.predict([X_ts[i]])[0]))
    plt.axis('off')
plt.show()
```

10. K-Means Clustering

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
data = load_breast_cancer()
X = data.data
```

```
X = StandardScaler().fit_transform(X)
```

```
k = KMeans(n_clusters=2)
y = k.fit_predict(X)
```

```
p = PCA(n_components=2)
X_p = p.fit_transform(X)
```

```
c = p.transform(k.cluster_centers_)
```

```
plt.scatter(X_p[:,0],X_p[:,1],c=y)
plt.scatter(c[:,0],c[:,1],c='red')
plt.show()
```